



ESCUELA POLITÉCNICA NACIONAL



Escuela de Formación de Tecnólogos
Tecnología Superior en Desarrollo de Software

PROYECTO SEGUNDO BIMESTRE CRUD DE MIEMBROS DEL GIMNASIO

Asignatura: Programación Profesor:
Ing. Lorena Chulde MSc.
Período Académico: 2026-A

Integrantes:

- Collaguazo Simbaña Fabián
- Cruz Piña Brayan Alexis

Quito – Ecuador
Julio 2026

RESUMEN

Este proyecto nació de una necesidad muy concreta que vimos en los gimnasios pequeños de nuestro entorno: muchos todavía llevan el control de sus miembros en cuadernos u hojas sueltas y eso, tarde o temprano, genera pérdidas de información, datos duplicados y un desorden difícil de manejar. Para aportar una solución sencilla, nosotros desarrollamos una aplicación de consola en lenguaje C++ que permite hacer las cuatro operaciones básicas de cualquier sistema de información: crear, consultar, actualizar y eliminar lo que se conoce como CRUD, por sus siglas en inglés. La particularidad es que no usamos una base de datos compleja, sino archivos de texto planos, esos que se pueden abrir con el bloc de notas, porque son fáciles de entender y más que suficientes para un volumen de datos como el de un gimnasio barrial.

Durante todo el proceso aplicamos los conceptos que fuimos aprendiendo en la materia de Programación que fue estructuras, funciones, manejo de archivos con fstream, validación de datos y creación de menús interactivos. El resultado final es un sistema funcional, que corre en la terminal y que realmente puede ayudar a organizar la información de un negocio real sin necesidad de instalar nada extraño. Creemos que, más allá de la nota, el proyecto nos dejó la experiencia de construir algo útil desde cero.

Palabras clave: CRUD, C++, archivos planos, gimnasio, gestión de miembros.

ABSTRACT

This project came from a real need we observed in small gyms: many still manage their member information manually, which leads to mistakes, lost data and a lot of disorganization. To offer a simple tool, we developed a console application using C++ that performs the four basic operations of any data system: create, read, update and delete (known as CRUD). Instead of a database, we used plain text files, because they are easy to work with and perfectly adequate for the number of records a small gym handles.

Throughout the development we applied everything we learned in our Programming course: structures, functions, file handling and data validation. The result is a working system that runs in the terminal and could genuinely help keep a real business organized. We feel that this project not only tested our programming skills but also showed us how to build something practical from scratch.

Keywords: CRUD, C++, gym, file handling, member management.

CAPÍTULO 1

DESCRIPCIÓN DEL PROYECTO

1.1 Descripción

Cuando arrancamos con la idea, nos pusimos en el lugar del dueño de un gimnasio que tiene que anotar a mano quién se inscribe, cuándo vence cada membresía o si alguien pidió cambiar su plan. Eso no solo toma tiempo, sino que casi siempre termina en errores, extravío de papeles o registros duplicados. Por eso decidimos crear un programa que hiciera ese trabajo de manera automática y ordenada.

Nuestro sistema es una aplicación de consola hecha en C++ que maneja la información de cada miembro mediante un menú fácil de seguir. La persona que lo use puede dar de alta a un nuevo miembro, ver la lista completa de todos los anotados, modificar los datos de alguien (por ejemplo, si se pasa de una membresía mensual a una trimestral) y también eliminar el registro de quien se va del gimnasio. Todo eso queda guardado en un archivo .txt, así que, aunque se cierre el programa, la información no se pierde y vuelve a aparecer la próxima vez que se ejecuta.

1.2 Objetivo General

Nos propusimos desarrollar una aplicación CRUD en C++ que permitiera administrar de forma sencilla y eficaz los datos de los miembros de un gimnasio, usando únicamente archivos de texto plano para el almacenamiento.

1.3 Objetivos Específicos

Para alcanzar el objetivo general del proyecto, se definieron varios objetivos específicos que guiaron el desarrollo del sistema:

- Diseñar una estructura de datos que permita almacenar de forma organizada toda la información relevante de cada miembro del gimnasio, evitando mezclar los diferentes campos de información.
- Implementar las cuatro operaciones fundamentales del CRUD (Crear, Leer, Actualizar y Eliminar), verificando el funcionamiento correcto de cada una de ellas de manera independiente.
- Desarrollar un sistema de almacenamiento mediante archivos de texto que permita guardar y recuperar la información de los miembros, garantizando la persistencia de los datos entre diferentes sesiones del programa.
- Validar los datos ingresados por el usuario para evitar registros con información incorrecta, como identificadores duplicados, campos vacíos o datos inconsistentes.
- Facilitar la administración de los miembros del gimnasio mediante una aplicación práctica, organizada y fácil de utilizar, que permita gestionar la información de manera eficiente.

1.4 Alcance

Desde el comienzo tuvimos claro hasta dónde queríamos llegar. El sistema permite registrar nuevos miembros, consultar los que ya están guardados, modificar cualquiera de sus datos y eliminar registros cuando sea necesario. Funciona completamente desde la consola de Windows y, como dijimos, guarda todo en un archivo .txt.

Para ser realistas y no terminar con un proyecto eterno, definimos lo que no íbamos a incluir. No implementamos inicio de sesión con usuarios y contraseñas, no conectamos el programa a una base de datos relacional, no hicimos una interfaz gráfica con ventanas ni

incluimos el control de pagos o alertas de vencimiento. Todo eso queda como posible mejora a futuro, pero el núcleo del sistema está completo y operativo.

1.5 Marco Teórico

Antes de escribir la primera línea de código, nos pusimos a repasar los conceptos que iban a sostener el proyecto.

El más importante es el modelo CRUD. Estas cuatro letras resumen las operaciones básicas que cualquier sistema de información debería poder hacer: crear nuevos registros, leer o consultar los existentes, actualizar datos cuando algo cambia y eliminar lo que ya no sirve. Básicamente, es el esqueleto de cualquier programa que maneje listas de personas, productos o cualquier otra cosa.

En cuanto al lenguaje, elegimos C++ porque es el que estábamos aprendiendo en la materia y porque nos da muchísimo control sobre lo que hace el programa, especialmente con el manejo de archivos. Si bien hay lenguajes más modernos, C++ nos obligó a pensar en cada detalle y eso ayudó a entender mejor cómo funcionan las cosas por dentro.

Sobre el almacenamiento, optamos por archivos planos. Un archivo de texto común, donde cada línea representa a un miembro o a un campo separado por algún carácter, es una solución muy simple pero efectiva para proyectos académicos. No necesitamos instalar un gestor de base de datos y, además, el archivo se puede revisar con el bloc de notas, lo cual nos vino mejor para las pruebas.

Por último, la base de datos de nuestro programa en realidad es una estructura (struct) que definimos con los atributos que consideramos indispensable un ID único para identificar a cada persona, el nombre completo, la edad, el plan contratado y la fecha de inscripción. Agrupar estos datos en una estructura nos permitió manipular la información de manera ordenada y pasarla entre funciones sin complicarnos.

CAPÍTULO 2

METODOLOGÍA

2.1 Metodología de Desarrollo

Como equipo, no seguimos una metodología rígida de esas que se usan en empresas grandes, pero sí nos organizamos por etapas para no pisarnos. Empezamos analizando el problema: hablamos con un conocido que trabaja en un gimnasio para entender qué datos eran importantes y cómo se manejaban actualmente. Luego diseñamos un esquema en papel de cómo sería el menú y qué funciones necesitaríamos.

La codificación la hicimos de a poco, función por función. Primero logramos que se pudiera guardar un miembro en el archivo; después programamos la lectura y la muestra en pantalla más tarde le sumamos la actualización y, por último, la eliminación. Cada avance lo probábamos en el momento. Cuando algo fallaba y falló muchas veces nos tocaba depurar, corregir y volver a probar hasta que todo funcionara como esperábamos. Fue un ciclo de error y aprendizaje constante, pero nos ayudó a tener un sistema sólido al final.

2.2 Herramientas utilizadas

Para el desarrollo usamos Visual Studio Code, que fue el entorno que más nos acomodamos y nos resultó muy cómodo para programar y depurar en C++. El código lo fuimos subiendo a un repositorio privado en GitHub, más que nada para tener un respaldo y para ver cómo cambiaba el proyecto con el tiempo. Como ya mencionamos, los datos se guardan en archivos .txt que el mismo programa genera y actualiza.

2.3 Diseño del Sistema

Al pensar en la experiencia de quien usa el programa, decidimos que todo girara alrededor de un menú principal. Apenas se ejecuta, aparece una pantalla con opciones

numeradas de registrar un nuevo miembro, listar todos, buscar por ID para modificar o eliminar, y la salida. Cada opción llama a una función independiente que se encarga de su tarea y luego regresa al menú. Así evitamos tener un código gigante y difícil de entender.

2.4 Estructuras utilizadas

El corazón de los datos es una estructura que elegimos en cada usuario. Dentro de ella guardamos cinco campos como el ID (un número entero que no se puede repetir), el nombre (una cadena de texto), la edad, el plan (Básico, Premium, VIP), la duración (mensual, trimestral, anual) y la fecha en que se inscribieron. Esta estructura nos permitió tratar cada registro como un paquete de información que podíamos pasar a las funciones de manera limpia.

2.5 Manejo de Archivos

Desde el principio supimos que el archivo de texto iba a ser donde guardaríamos la información. Decidimos que cada vez que se inicia el programa, este lee todo el archivo y carga los miembros en un arreglo o vector. Luego, cuando el usuario hace un cambio ya sea agregar, modificar o eliminar, el programa reescribe el archivo completo con los datos actualizados. No es la forma más eficiente del mundo, pero para la cantidad de registros que manejaría un gimnasio chico funciona perfecto y nos evitó dolores de cabeza.

CAPÍTULO 3

RESULTADOS

Registro de miembros

La primera función que sacamos adelante fue la de registrar. Cuando el usuario elige esa opción, el sistema pide uno por uno los datos ID, nombre, edad, plan, duración de la membresía y la fecha de inscripción. Si el ID ya existe, salta un aviso y no deja continuar

hasta que se ingrese uno distinto. Una vez aceptados, el nuevo miembro se suma a la lista y el archivo se actualiza.

```
===== Administrador =====
1.- Registrar miembro
2.- Mostrar miembro
3.- Buscar miembro
4.- Actualizar miembro
5.- Eliminar miembro
6.- Salir
Ingrese una opcion: 1
Ingrese el codigo: 1
Ingrese el nombre: Juan
Ingrese la edad: 20
-----Plan-----
1.- Basico
2.- Premiun
3.- VIP
Elija un plan: 1
-----Duracion-----
1.- Mensual
2.- Trimestral
3.- Anual
Elija la duracion: 1
Ingrese la fecha de inscripcion: 23/7/2026
Miembro Registrado correctamente
```

Consulta de miembros

La opción de consulta recorre todos los registros que están en memoria y los muestra en pantalla de forma ordenada. Al principio nos pasaba que, si había muchos miembros, la lista se veía amontonada, así que le agregamos algunos espacios y separadores para que fuera legible. La verdad, ver toda la información bien organizada fue una de las primeras satisfacciones del trabajo.

```
===== Administrador =====
1.- Registrar miembro
2.- Mostrar miembro
3.- Buscar miembro
4.- Actualizar miembro
5.- Eliminar miembro
6.- Salir
Ingrese una opcion: 2
---- Lista De Miembros ----
ID: 1
Nombre: Juan
Edad: 20
Plan: 1
Duracion: 1
Fecha de Inscripcion: 23/7/2026
-----
ID: 2
Nombre: Brayan
Edad: 20
Plan: 3
Duracion: 2
Fecha de Inscripcion: 12/24/2026
-----
ID: 3
Nombre: Carlos
Edad: 42
Plan: 3
Duracion: 3
Fecha de Inscripcion: 6/12/2026
-----
```

Modificación de registros

Para modificar, el usuario ingresa el ID del miembro que quiere cambiar. El sistema busca ese ID, muestra los datos actuales y pregunta qué campo se desea modificar. Nos costó un poco manejar el caso de que el ID no existiera, pero al final logramos que mostrara un mensaje claro y volviera al menú sin romperse. Una vez hechos los cambios, el archivo se actualiza de inmediato.

```
===== Administrador =====
1.- Registrar miembro
2.- Mostrar miembro
3.- Buscar miembro
4.- Actualizar miembro
5.- Eliminar miembro
6.- Salir
Ingrese una opcion: 4
Ingrese el id de miembro: 1
Nuevo nombre: Jose
Nueva edad: 25
Nuevo plan: 2
Nueva duracion: 3
Nueva fecha de inscripcion: 21/7/2026
Miembro actualizado Correctamente
```

Eliminación de miembros

La eliminación fue la función que más respeto nos dio, porque un error podía borrar la información equivocada. Pero logramos solucionarlo y una vez después de que el usuario elige la opción y escribe el ID, el sistema elimina los datos completos de ese registro. Una vez eliminado el miembro el sistema borra y se reescribe el archivo sin ese miembro.

```
===== Administrador =====
1.- Registrar miembro
2.- Mostrar miembro
3.- Buscar miembro
4.- Actualizar miembro
5.- Eliminar miembro
6.- Salir
Ingrese una opcion: 5
Ingrese el codigo a eliminar: 1
Miembro eliminado correctamente
```

Validación de datos

Al hacer las pruebas nos aseguramos de que el sistema se comportara correctamente ante situaciones inesperadas. Comprobamos que no dejara ingresar un ID duplicado, que no aceptara campos vacíos, que la modificación alterara solo el dato elegido y no todo el

registro, y que al eliminar a alguien no se borrarán otros miembros por error. También verificamos que, después de cerrar y volver a abrir el programa, todos los datos siguieran ahí.

```
===== Administrador =====
1.- Registrar miembro
2.- Mostrar miembro
3.- Buscar miembro
4.- Actualizar miembro
5.- Eliminar miembro
6.- Salir
Ingrese una opcion: 7
Opcion Invalida
```

```
===== Administrador =====
1.- Registrar miembro
2.- Mostrar miembro
3.- Buscar miembro
4.- Actualizar miembro
5.- Eliminar miembro
6.- Salir
Ingrese una opcion: 1
Ingrese el codigo: 9
Ingrese el nombre: Juan
Ingrese la edad: -21
Debe ser mayor que 0
```

```
===== Administrador =====
1.- Registrar miembro
2.- Mostrar miembro
3.- Buscar miembro
4.- Actualizar miembro
5.- Eliminar miembro
6.- Salir
Ingrese una opcion: 3
Ingrese el id del miembro: 6
ID incorrecto
```

```
===== Administrador =====
1.- Registrar miembro
2.- Mostrar miembro
3.- Buscar miembro
4.- Actualizar miembro
5.- Eliminar miembro
6.- Salir
Ingrese una opcion: 4
Ingrese el id de miembro: 6
No se encontro al miembro
```

```
===== Administrador =====
1.- Registrar miembro
2.- Mostrar miembro
3.- Buscar miembro
4.- Actualizar miembro
5.- Eliminar miembro
6.- Salir
Ingrese una opcion: 5
Ingrese el codigo a eliminar: 6
No se encontro el libro
```

CONCLUSIONES

Llegar al final de este proyecto nos dejó un montón de aprendizajes. En primer lugar, aplicar los conceptos de programación en C++ a un caso real y no a ejercicios sueltos nos ayudó a entender de verdad cómo se relacionan las estructuras, las funciones y el manejo de archivos. Construir un sistema CRUD desde cero nos mostró el ciclo completo de la información: cómo nace un dato, cómo se consulta, cómo se modifica y cómo se elimina sin dejar cabos sueltos.

El uso de archivos planos, que al principio nos parecía algo muy sencillo, demostró ser una alternativa perfectamente válida para proyectos académicos y para resolver necesidades concretas sin agregar complejidad innecesaria. Además, nos obligó a ser cuidadosos con la lectura y escritura, lo que nos dio una base sólida para después saltar a bases de datos si quisiéramos.

También aprendimos que la organización del código importa muchísimo. Dividir el programa en funciones independientes, cada una con una tarea clara, nos salvó de dolores de cabeza cuando algo fallaba. Y poder validar lo que el usuario ingresa no es un lujo, es una necesidad, si no lo hacíamos, cualquier error de tipeo podía dejar el sistema inconsistente.

Creemos que el sistema cumple con los objetivos que nos fijamos y que realmente podría ser usado por un gimnasio pequeño para llevar el control de sus miembros de manera ordenada y sin papeles.

RECOMENDACIONES

Aunque estamos contentos con el resultado, somos conscientes de que se podrían hacer muchas mejoras si esto se llevara a un entorno más profesional. Lo primero que recomendaríamos es migrar el almacenamiento a una base de datos como SQLite, que sigue siendo liviana, pero permite manejar mucha más información y hacer consultas más avanzadas. También sería bueno agregar un sistema de inicio de sesión para que solo el administrador pueda modificar datos, porque ahora mismo cualquiera que abra el programa puede hacer cambios.

Otro paso lógico sería desarrollar un módulo de pagos, donde se registre cuándo paga cada miembro, cuándo vence su membresía y que el sistema avise con anticipación los vencimientos. Y para que el programa sea más amigable, se podría crear una interfaz gráfica con ventanas y botones, aunque eso ya implicaría meterse con otras librerías.

Por último, nos parece que una función muy útil sería la generación de reportes automáticos, por ejemplo, una lista de miembros activos, otra de inactivos o un conteo de cuántos hay por cada tipo de membresía. Eso le daría al dueño del gimnasio información valiosa sin tener que contarlos a mano.

REFERENCIAS BIBLIOGRÁFICAS

- Deitel, P., & Deitel, H. (2020). *C++ How to Program* (10.^a ed.). Pearson.
- Stroustrup, B. (2014). *Programming: Principles and Practice Using C++* (2.^a ed.). Addison-Wesley.
- Programiz. (s. f.). *C++ File Handling*.
<https://www.programiz.com/cppprogramming/files>
- cplusplus.com. (s. f.). *Input/Output with Files*.
<https://cplusplus.com/doc/tutorial/files/>